DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING

Implementation of Bayesian Networks

ARTIFICIAL INTELLIGENCE LAB
CSE 316

GREEN UNIVERSITY OF BANGLADESH

November 16, 2025

1 Learning Objectives

- Learn how Bayesian Networks represent uncertain relationships between variables using probability.
- Implement basic Bayesian inference algorithms such as Variable Elimination and Sampling.
- Learn how to test and evaluate Bayesian models using posterior probability and likelihood.

2 Introduction for Beginners

What is a Bayesian Network?

A **Bayesian Network (BN)** is a special kind of graph that helps us reason about uncertainty. It connects causes and effects using probabilities. Each node represents a random variable (like "Disease" or "Symptom"), and each edge shows a direct relationship or influence.

Think of it like this: If a person smokes, they may have a higher chance of getting lung cancer, which might cause coughing or fatigue. In a Bayesian Network, this chain of cause and effect can be represented clearly and calculated mathematically.

Main Parts of a Bayesian Network:

- **Nodes:** Represent variables or events (e.g., Smoking, Cancer, Coughing).
- **Edges:** Show relationships (e.g., Smoking causes Cancer).
- **Conditional Probability Tables (CPTs):** Define how likely one event is, given another.

Bayesian Networks benefits:

- They help us predict unknown information from known evidence.
- They allow us to reason about causes and effects in uncertain situations.
- They can update beliefs automatically when new data appears.

A Simple Analogy: Imagine a detective solving a mystery:

- Evidence: footprints, fingerprints.
- Hidden causes: who the suspect might be.

Every time the detective finds a new clue, their belief about the suspect changes. This is exactly how Bayesian reasoning works.

3 Theoretical Foundations of Bayesian Networks

A Bayesian Network represents a joint probability distribution over several random variables. If we have variables $X_1, X_2, \ldots, X_n$, then:

$$P(X_1, X_2, \ldots, X_n) = \prod_{i=1}^{n} P(X_i \mid Parents(X_i))$$

This formula means: Each variable depends only on its parent variables in the graph — making the computation efficient.

3.1 Key Components

3.1.1 Structure (Graph Representation)

A **Directed Acyclic Graph (DAG)** shows how the variables are connected. For example:

$$Smoking \rightarrow LungCancer \rightarrow Coughing$$

 © Dept. of Computer Science and Engineering, GUB

3.1.2 Conditional Probability Tables (CPTs)

Each node has a table that describes how likely it is to be true given its parent nodes.

Lung Cancer	Smoking	P(Lung Cancer — Smoking)
True	True	0.08
True	False	0.01
False	True	0.92
False	False	0.99

3.1.3 Independence

A key strength of Bayesian Networks is conditional independence:

$$P(A, B \mid C) = P(A \mid C) \cdot P(B \mid C)$$

This property allows complex systems to be represented with fewer parameters.

4 Inference in Bayesian Networks

Inference means finding the probability of something we care about based on known evidence. For example:

$$P(Disease \mid Symptom)$$

tells us how likely a disease is, given that we have observed a symptom.

4.1 Exact Inference: Variable Elimination

Variable Elimination is a systematic way to compute these probabilities:

1. Choose an order to eliminate unnecessary variables.
2. Multiply relevant factors.
3. Sum out hidden variables and normalize the results.

4.2 Approximate Inference: Sampling Methods

When exact calculation is too complex, we use random sampling techniques:

- **Monte Carlo Sampling:** Estimate probabilities by repeated random trials.
- **Likelihood Weighting:** Focuses on samples consistent with evidence.
- **Gibbs Sampling:** Iteratively samples each variable based on its neighbors.

5 Example Problem: Traditional Burglary Bayesian Network

5.1 Scenario Description

Consider the classic **Burglary–Earthquake–Alarm** problem — a well-known example for Bayesian Networks illustrating how multiple causes can influence a common effect. A home security system is designed to detect burglaries. However, the alarm can also go off because of an earthquake. When the alarm sounds, two neighbors — **John** and **Mary** — may call the homeowner, depending on whether they heard it. We aim to model this uncertain relationship and perform probabilistic inference, such as finding:

$$P(Burglary \mid JohnCalls = True, MaryCalls = True)$$

which expresses the probability of a burglary having occurred given that both John and Mary report hearing the alarm.

 © Dept. of Computer Science and Engineering, GUB

5.2 Problem Goals

1. Construct a Bayesian Network to represent the relationships between burglary, earthquake, alarm, and phone calls.
2. Encode the conditional probability tables (CPTs) that define the relationships.
3. Perform probabilistic inference to compute posterior probabilities.
4. Learn parameters from synthetic data and evaluate the model using performance metrics.

5.3 Bayesian Network Structure

The network includes five binary random variables:

Variable	Meaning
B	Burglary occurs (True/False)
E	Earthquake occurs (True/False)
A	Alarm goes off (True/False)
J	John calls (True/False)
M	Mary calls (True/False)

5.4 Graphical Model Representation

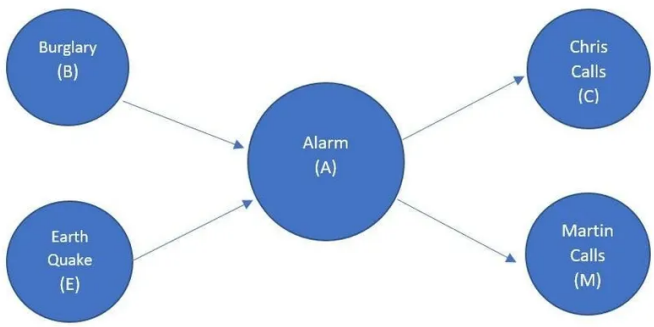


Figure 1: Structure of the Traditional Burglary Bayesian Network.

5.5 Mathematical Factorization

The joint probability distribution is factored as:

$$P(B, E, A, J, M) = P(B) \times P(E) \times P(A \mid B, E) \times P(J \mid A) \times P(M \mid A)$$

This expresses how each variable depends only on its direct parents in the network.

5.6 Query Example

Given evidence from the neighbors, we can compute:

$$P(Burglary = True \mid JohnCalls = True, MaryCalls = True)$$

using exact or approximate inference methods (Variable Elimination, Likelihood Weighting, Gibbs Sampling).

The next section provides a complete Python-based implementation of this model, step-by-step, using the `pgmpy` library.

 © Dept. of Computer Science and Engineering, GUB

6 Implementation Guide (Step-by-step)

The following implementation uses `pgmpy`. The code blocks are ready for Google Colab (Python) style and include explicit places for output.

Step 0: Environment & Libraries

```
#Author: JMH
#Install dependencies
!pip install pgmpy==0.1.24 pydot networkx matplotlib pandas scikit-learn --quiet
#Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import networkx as nx

from pgmpy.models import BayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
from pgmpy.sampling import BayesianModelSampling, GibbsSampling
from pgmpy.estimators import MaximumLikelihoodEstimator, BayesianEstimator
from sklearn.model_selection import train_test_split

print("Libraries imported successfully.")
```

Output Libraries imported successfully.

Step 1: Define the BN structure

```

model = BayesianNetwork([('B','A'), ('E','A'), ('A','J'), ('A','M')])
print("Edges in the model:", model.edges())
```

Output Edges in the model: [('B', 'A'), ('A', 'J'), ('A', 'M'), ('E', 'A')]

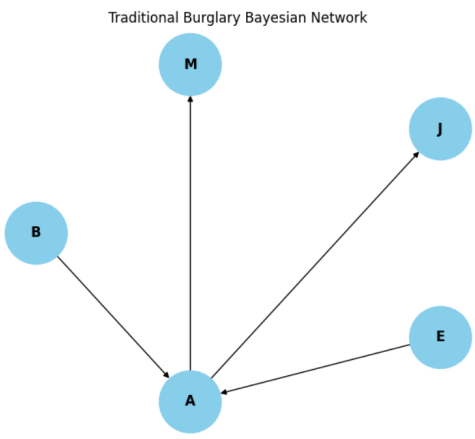
Step 2: Specify CPTs (manual values for the canonical burglary problem)

```
#Define Conditional Probability Tables
cpd_B = TabularCPD('B', 2, [[0.999], [0.001]])
cpd_E = TabularCPD('E', 2, [[0.998], [0.002]])

cpd_A = TabularCPD(
    'A', 2,
    [[0.999, 0.71, 0.06, 0.05],
     [0.001, 0.29, 0.94, 0.95]],
    evidence=['B', 'E'], evidence_card=[2, 2]
)

cpd_J = TabularCPD(
    'J', 2,
    [[0.95, 0.10],
     [0.05, 0.90]],
    evidence=['A'], evidence_card=[2]
)

cpd_M = TabularCPD(
    'M', 2,
    [[0.99, 0.30],
     [0.01, 0.70]],
    evidence=['A'], evidence_card=[2]
)
model.add_cpds(cpd_B, cpd_E, cpd_A, cpd_J, cpd_M)
print("Model check:", model.check_model())
```

 © Dept. of Computer Science and Engineering, GUB

Step 3: Visualize the network

```
#Visualize the network structure
if model and len(model.nodes()) > 0:
    plt.figure(figsize=(6,5))
    try:
        pos = nx.shell_layout(model)
        nx.draw(model, pos, with_labels=True, node_size=3000, node_color="skyblue",
                font_size=12, font_weight='bold')
        plt.title("Traditional Burglary Bayesian Network")
        plt.show()
    except Exception as e:
        print(f"Error drawing the network: {e}")
else:
    print("Model is empty or invalid. Cannot draw the network.")
```



Output

Step 4: Exact inference (Variable Elimination)

```
#Exact Inference (Variable Elimination)
infer = VariableElimination(model)
q = infer.query(variables=['B'], evidence={'J':1, 'M':1})
print("P(Burglary | JohnCalls=True, MaryCalls=True):\n", q)
```

Output:

```
P(Burglary | JohnCalls=True, MaryCalls=True):
 +------+----------+
| B    |   phi(B) |
+======+==========+
| B(0) |   0.7158 |
+------+----------+
| B(1) |   0.2842 |
+------+----------+
```

Step 5: Generate synthetic data and parameter learning

```
# Sampling synthetic data from the model
sampler = BayesianModelSampling(model)
data = sampler.forward_sample(size=5000, seed=42)

# Pgmpy returns boolean-like (0/1) values. Convert to int:
data = data.astype(int)
display(data.head())

# Generate Synthetic Data and Parameter Learning
```

 © Dept. of Computer Science and Engineering, GUB

```python

sampler = BayesianModelSampling(model)
data = sampler.forward_sample(size=5000, seed=42)
data = data.astype(int)
display(data.head())

# Split dataset for training and testing
train, test = train_test_split(data, test_size=0.2, random_state=42)

# Learn parameters using Maximum Likelihood Estimation (MLE)
model_learned = BayesianNetwork([('B','A'), ('E','A'), ('A','J'), ('A','M')])
model_learned.fit(train, estimator=MaximumLikelihoodEstimator)

print("Learned CPTs:")
for cpd in model_learned.get_cpds():
    print(cpd)
```

Output:

```
Learned CPTs:
+------+---------+
| B(0) | 0.99925 |
+------+---------+
| B(1) | 0.00075 |
+------+---------+
+------+----------------------+-------------------+------+------+
| B    | B(0)                 | B(0)              | B(1) | B(1) |
+------+----------------------+-------------------+------+------+
| E    | E(0)                 | E(1)              | E(0) | E(1) |
+------+----------------------+-------------------+------+------+
| A(0) | 0.9992477432296891   | 0.4444444444444444 | 0.0  | 0.5  |
+------+----------------------+-------------------+------+------+
| A(1) | 0.0007522567703109327 | 0.5555555555555556 | 1.0  | 0.5  |
+------+----------------------+-------------------+------+------+
+------+---------+
| E(0) | 0.99775 |
+------+---------+
| E(1) | 0.00225 |
+------+---------+
```

```
+------+----------------------+---------------------+
| A    | A(0)                 | A(1)                |
+------+----------------------+---------------------+
| J(0) | 0.9473552268739033   | 0.18181818181818182 |
+------+----------------------+---------------------+
| J(1) | 0.052644773126096764 | 0.8181818181818182  |
+------+----------------------+---------------------+
+------+----------------------+--------------------+
| A    | A(0)                 | A(1)               |
+------+----------------------+--------------------+
| M(0) | 0.990975181749812    | 0.2727272727272727 |
+------+----------------------+--------------------+
| M(1) | 0.009024818250188017 | 0.7272727272727273 |
+------+----------------------+--------------------+
```

Step 6: Approximate inference

6.1 Likelihood Weighting

```python
# Likelihood-Weighted Sampling (Approx. Inference)
from pgmpy.sampling import BayesianModelSampling

sampler = BayesianModelSampling(model)

samples = sampler.likelihood_weighted_sample(evidence=[('J', 1), ('M', 1)], size=2000)

# Compute weighted probability P(B=1 | J=1, M=1)
weighted_prob = (
    samples[samples['B'] == 1]['_weight'].sum() / samples['_weight'].sum()
)
print(f"Estimated P(B=1 | J=1, M=1) [Likelihood Weighted Sampling]: {weighted_prob:.5f}")
```

Output: Estimated P(B=1 — J=1, M=1) [Likelihood Weighted Sampling]: 0.15190

6.2 Gibbs Sampling (MCMC)

```python
# Gibbs sampling using pgmpy
from pgmpy.sampling import GibbsSampling

g_sampler = GibbsSampling(model)

# Perform a short "burn-in" phase manually by discarding some initial samples
_ = g_sampler.sample(size=500)   # discard these samples (warm-up)

g_samples = g_sampler.sample(size=5000)

g_subset = g_samples[(g_samples['J'] == 1) & (g_samples['M'] == 1)]
if len(g_subset) > 0:
```

 © Dept. of Computer Science and Engineering, GUB

```
    gibbs_estimate = g_subset[g_subset['B'] == 1].shape[0] / len(g_subset)
    print(f" Estimated P(B=1 | J=1, M=1) [Gibbs Sampling]: {gibbs_estimate:.5f}")
else:
    print(" No samples with J=1, M=1 found, increase sample size.")
```

Output: Estimated P(B=1 — J=1, M=1) [Gibbs Sampling]: 0.50000

Step 7: Evaluation Metrics

1. In a Bayesian Network, the **log-likelihood** measures how well the network explains the data. For a dataset D with N samples and variables $X_1, \ldots, X_n$, it is:

$$\log L(\theta; D) = \sum_{j=1}^{N} \sum_{i=1}^{n} \log P\big(x_i^{(j)} \mid Pa(X_i^{(j)}), \theta_i\big)$$

It sums the log probabilities of each variable given its parents, with higher values indicating a better fit to the data. 2. **Posterior Accuracy** measures how well the predicted posterior probabilities match the true outcomes. For N samples, it is:

$$\text{Accuracy} = \frac{1}{N} \sum_{j=1}^{N} \mathbf{1}\big(\hat{y}^{(j)} = y^{(j)}\big), \quad \hat{y}^{(j)} = \arg\max_{y} P(Y = y \mid X^{(j)})$$

```
# Evaluation: Log-Likelihood Computation
import numpy as np

def log_likelihood(model, df):
    ll = 0.0
    N = len(df)

    # Iterate through each instance
    for _, row in df.iterrows():
        prob = 1.0
        for cpd in model.get_cpds():
            var = cpd.variable
            evidence = cpd.get_evidence()
            if not evidence:
                # No parents
                prob *= cpd.values[row[var]]
            else:
                # Find index corresponding to evidence states
                parent_states = tuple(row[parent] for parent in evidence)
                # pgmpy stores CPTs in lexicographic order
                prob *= cpd.get_value(**{var: row[var], **dict(zip(evidence,
    parent_states))})
        ll += np.log(prob + 1e-12)  # avoid log(0)
    return ll / N  # average log-likelihood

# Compute and print average log-likelihood
ll_value = log_likelihood(model_learned, test)
print(f"Average Log-Likelihood on test data: {ll_value:.5f}")
```

Output: Average LogLikelihood on test data: -0.28008

```
# Posterior accuracy: predicting B given J, M
correct, total = 0, 0
infer_learned = VariableElimination(model_learned)
for _, row in test.iterrows():
    q = infer_learned.map_query(variables=['B'], evidence={'J':int(row['J']), 'M':int(row
    ['M'])})
    if q['B'] == row['B']:
        correct += 1
    total += 1

accuracy = correct / total
print(f"Posterior prediction accuracy: {accuracy*100:.2f}%")
```

Output: Posterior prediction accuracy: 100.00%

 © Dept. of Computer Science and Engineering, GUB

Notebook Link: https://colab.research.google.com/drive/1oq7PDND$_AJK_GMKvlflHS76RJig7R9hA?usp = sharing$

7 Lab Exercises

Exercise 1: Rain and Sprinkler

Problem: Compute the probability of rain given that the grass is wet.

Variables:

- Rain (R): Yes/No
- Sprinkler (S): Yes/No
- Wet Grass (W): Yes/No

Sample CPTs:

$P(R)$	
R=Yes	0.2
R=No	0.8

$P(S \mid R)$	R=Yes	R=No
S=Yes	0.01	0.4
S=No	0.99	0.6

$P(W \mid S, R)$	...	...
W=Yes	0.99	0.9
W=No	0.01	0.1

Task: Compute $P(R = \text{Yes} \mid W = \text{Yes})$ and log-likelihood for a dataset.

Exercise 2: Medical Diagnosis

Problem: Compute the probability that a patient has a disease given a positive test and symptom.

Variables:

- Disease (D): Yes/No
- Symptom1 (S1): Yes/No
- Symptom2 (S2): Yes/No
- Test Result (T): Positive/Negative

Sample CPTs:

$P(D)$	
D=Yes	0.01
D=No	0.99

$P(S1 \mid D)$	D=Yes	D=No
S1=Yes	0.8	0.1
S1=No	0.2	0.9

$P(T \mid D)$	D=Yes	D=No
T=Positive	0.9	0.05
T=Negative	0.1	0.95

Task: Compute $P(D = \text{Yes} \mid S1 = \text{Yes}, T = \text{Positive})$ and evaluate posterior accuracy.

© Dept. of Computer Science and Engineering, GUB